

de transporte (context/1 , dispatch/3 com um responder), e o Vapor.Plug é um responder sobre Plug.Conn . TLS, HTTP/2, autenticação e telemetria passam a ser os do endpoint da aplicação. Phoenix feito, via Plug forward /llm, Vapor.Plug, name: MyApp.LLM . O contexto (índice do vocabulário para a decodificação restrita, que é caro) é montado uma vez, na árvore de supervisão. O Phoenix não precisa de mais nada. LiveView gancho no núcleo + receita A dor real é mostrar um agente trabalhando ao vivo. Vapor.Agent.run(, on event: fn e, - end) entrega cada evento do diário no momento em que é gravado. Os tokens de geração já chegam como

mensagens ao processo que pediu (:vapor, ref, :token,), e uma LiveView é um processo. A receita está abaixo. Ecto behaviour no núcleo + esboço Vapor.Agent.Store define o contrato (gravar o evento seq com exclusividade; carregar só o prefixo que verifica). Store.File o implementa com OTP puro e está testado. Um adaptador Ecto/Postgres obtém a exclusividade de UNIQUE (run id, seq) . O esboço está abaixo; não foi executado aqui por falta de Postgres. Oban o núcleo já entrega o contrato; Oban agenda Os workflows duráveis do Oban resolvem retomar depois de cair. Com Store + resume/4 , a retomada vem do diário: repete-se o que foi gravado,